● 진행 중 · 2026.09 -

다중 로봇 협업 스마트팜

파프리카를 살피고, 수확하고, 옮기고, 선별하는 농작업 자동화 프로젝트입니다.

예찰·수확·운송 로봇과 선별 장치를 ROS 2로 연결합니다. 중앙 관제가 각 로봇의 작업과 상태를 관리하며, 예찰 결과를 다음 수확 작업으로 이어주는 시스템을 개발하고 있습니다.

나의 역할

예찰 로봇 개발

재배 구역을 먼저 주행하며 수확할 위치와 예상 수확량을 정리하고, 관제와 수확 로봇이 사용할 작업 정보를 만드는 역할입니다.

담당 업무 · 설계 및 개발 중

- **자율주행 · 복귀 · 도킹**
  Nav2 기반 예찰 경로 이동과 임무 종료 후 스테이션 복귀·도킹을 담당합니다.
- **수확 작업 정보 생성**
  AI 탐지 결과와 로봇 위치를 연결해 작업 위치·순서·예상 수확량을 구성합니다.
- **임무 상태 · 관제 연동**
  예찰 FSM과 ROS 2 통신 구조를 설계하고, 작업 진행·완료·로봇 상태를 관제에 전달합니다.



재배 환경 구성 이미지

전체 프로젝트 목표

예찰에서 선별까지 하나의 작업 흐름으로

예찰 결과 → 수확 작업
수확물 인계 → 운송·선별



실제 로봇·테스트 환경

01 **예찰** 내 담당 · 주행 및 작업 정보	02 **수확** 로봇팔로 작물 수확	03 **운송** 수확물 이송	04 **선별** 분류·후속 공정

ROS 2 · Nav2 · FSM · Action / Topic · 7인 팀

예찰 로봇에서 제가 하는 일

"어디를 이동하고, 어디서 몇 개를 수확할지"를 다음 로봇이 사용할 정보로 정리합니다.

담당 동작 흐름

01 관제 명령을 받아 예찰 경로 주행

목표 위치를 Nav2에 전달하고, 재배 구역의 예찰 경로를 따라 이동하도록 구성합니다.

02 작물 탐지 위치를 수확 작업 범위로 변환

AI 최초 탐지 시점의 로봇 위치와 로봇팔의 유효 작업 폭을 이용해 작업 범위를 계산합니다.

03 작업 위치 · 순서 · 수확량 확정

범위의 중앙을 작업 위치로 잡고 예상 수확량을 붙여 Harvest Task를 만듭니다. 수확량이 0이면 작업 생성은 생략하고 검사 완료를 보고합니다.

04 진행 상황 보고, 예찰 종료 후 복귀

구간별 완료는 Feedback, 전체 임무 종료는 Result로 구분합니다. 복귀·도킹까지 상태 흐름을 연결합니다.

설계한 핵심 구조



FSM으로 동작 책임 분리

대기 → 주행 → 작업 범위 확정 → 수확량 확정 → 구역 완료 → 도킹

ROS 2 인터페이스 설계

Action은 예찰·복귀 임무, Topic은 위치·상태 보고에 사용하도록 역할을 나눴습니다.

● 현재 진행 상태

요구사항·FSM·작업 데이터·통신 구조를 문서화했습니다. 주행, AI 연동, 작업 생성, 복귀·도킹의 구현과 통합 검증을 이어가는 프로젝트입니다.

2026.09.17 기준 · 실측 성능과 세부 파라미터는 검증 후 확정

2026.05, 08 - 09 · 4인 팀

POCO · AI 자세 코칭 모니터암

사용자의 자세를 분석하고, 모니터의 위치와 기울기를 조절하는 온디바이스 AI 시스템입니다.

Raspberry Pi에서 카메라 영상으로 자세를 판단하고, 4축 모니터암을 제어합니다. 사용자는 화면에서 보정·측정·제어를 수행하고 자세 기록을 확인할 수 있습니다.

나의 역할

짐벌 · 프론트 · 멀티프로세싱 · 코드 통합

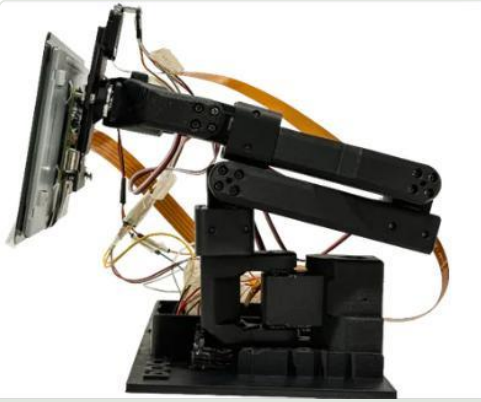
모니터의 기울기를 보정하는 짐벌 제어와 사용자 화면을 구현하고, 병렬 실행 구조와 형상 관리·코드 병합을 담당했습니다.

직접 구현한 내용

- **짐벌 구현**
  IMU 기울기를 이용해 모니터 자세를 보정하는 Motor 3·4 제어를 구현했습니다.
- **프론트엔드 구현**
  사용자 조작, 보정, 실시간 상태 표시와 자세 기록·리포트 화면을 구성했습니다.
- **멀티프로세싱 · 형상 관리**
  비전과 하드웨어 실행을 분리하고, 팀원별 코드를 병합해 통합 실행 구조를 관리했습니다.



정면 · 실제 제작물



측면 · 4축 모니터암

모니터암 안에서의 내 담당

Motor 3·4 · 짐벌 자세 보정

사용자 위치에 맞추는 전체 모니터암 중,
IMU 기반 수평·기울기 보정 제어를 담당했습니다.

화면 ↔ AI ↔ 하드웨어

프론트와 실행 프로세스를 연결하고, 병합된 코드가 하나의 시스템으로 동작하도록 통합했습니다.

01	02	03	04
카메라 입력 사용자의 자세 촬영	**AI 자세 판단** 로컬에서 자세 분석	**모니터암 제어** 위치 조절 · 짐벌 보정	**화면 · 기록** 상태 확인 · 리포트

Python · Raspberry Pi 5 · IMU / PID · PyQt · Shared Memory · Git / GitHub

GitHub ↗ 개발 보고서 ↗ 시연 영상 ↗

직접 구현한 네 가지 영역

짐벌 제어와 화면을 구현하고, 여러 기능이 동시에 동작하도록 실행 구조와 코드를 통합했습니다.

짐벌 제어 01

모니터의 기울기를 보정하는 제어 루프

IMU 기울기 → 목표 대비 오차 → PID · 모터 보정

IMU에서 현재 기울기를 읽고 목표 자세와의 오차를 계산해 Motor 3·4를 제어했습니다. 움직이는 모니터암에서 모니터의 수평·기울기를 유지하는 역할입니다.

프론트엔드 02

사용자가 보정하고 상태를 확인하는 화면

프로필 · 설정 | 보정 · 측정 | 실시간 상태 | 기록 · 리포트

PyQt 기반 조작·상태 화면과 Streamlit·Plotly 기반 자세 리포트를 구현했습니다. 사용자 입력과 실행 결과가 화면에서 이어지도록 연결했습니다.

멀티프로세싱 03

비전 처리와 하드웨어 제어의 병렬 실행

카메라 · 비전 ↔ 공유 메모리 · 큐 ↔ UI · 하드웨어

프로세스 간 데이터를 공유하고, 상태값과 제어 이벤트를 구분했습니다. 오래된 프레임이 쌓이면 최신 프레임을 우선 소비하도록 개선했습니다.

자세 판단 지연 ~~약 1.35초~~ → **약 50ms**

형상 관리 · 코드 병합 04

파트별 구현을 하나의 실행 시스템으로 통합

파트별 변경 → 코드 병합 → 통합 실행 확인

Git/GitHub로 코드 변경을 관리하고 팀원별 구현을 병합했습니다. 프론트·비전·하드웨어 간 연결을 맞추며 통합 코드를 관리했습니다.

지연 수치: 개발완료보고서 p.13 내부 측정. Overrun 발생 후와 최신 프레임 소비 적용 후를 비교한 Pose 경로 기준입니다. 현재 기본 모드는 POSE_ONLY이며 Face 피로도 분석은 비활성화 상태입니다.

2026.07.23 - 07.30 · 4인 팀

KICK-CAN · 분산 제어 RC카

무선 조종기와 차량 내부 CAN 네트워크를 연결해 실제 차량의 전장 구조를 구현한 프로젝트입니다.

조종기, 주행 제어, 센서, 표시·경고를 4개 노드로 나눴습니다. 조종기 명령은 Bluetooth로 차량에 전달하고, 차량 내부에서는 CAN으로 제어·센서·상태 데이터를 공유합니다.

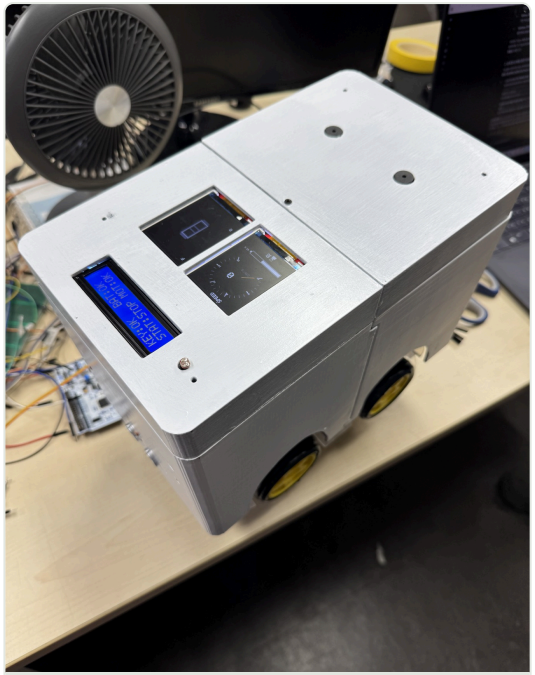
나의 역할

Controller 하드웨어 · 펌웨어 개발

사용자 인증부터 조이스틱·스위치 입력, 명령 패킷 생성과 무선 송신까지 STM32F411 조종기 전체 흐름을 담당했습니다.

직접 구현한 내용

- **RFID 인증 · 전원 인터록**
  등록 카드 인증 전에는 주변장치 전원과 제어 명령 송신을 차단했습니다.
- **조이스틱 · 스위치 입력 처리**
  ADC/DMA로 값을 수집하고 필터·중립 구간·디바운싱으로 입력을 안정화했습니다.
- **명령 생성 · Bluetooth 송신**
  차량과 합의한 패킷을 만들고, 인증·전원 상태를 확인한 뒤 HC-05로 전달했습니다.



전체 차량 · 팀 제작물



Controller · 내 담당

내 담당
Controller
인증 · 입력 · 패킷

Bluetooth
UART / HC-05
→

차량
Main Node
4WD 주행 제어

CAN Bus
↔

차량
Node 1 · Node 2
센서 · 표시 · 경고

STM32F411 | C / HAL | FreeRTOS | ADC / DMA | SPI | UART / Bluetooth

Controller GitHub ↗ 발표·시연 자료 ↗

조작이 차량 명령이 되기까지

인증된 사용자에게만 제어를 허용하고, 실제 하드웨어의 입력 흔들림을 처리했습니다.

01 **RFID 인증** 카드 UID 확인 → 02 **전원 허용** 릴레이 활성화 → 03 **입력 안정화** 조이스틱 · 스위치 → 04 **패킷 생성** 명령 ID · 데이터 → 05 **무선 송신** Main Node로 전달

구현 01 · 인증과 전원 제어

인증 전

제어 잠금

STM32·RFID만 동작
주변장치 전원 OFF
제어 패킷 송신 차단

등록 카드 인증 후

제어 허용

릴레이로 주변장치 ON
입력 처리 시작
정상 상태에서 송신

MFRC522를 SPI로 연결해 UID를 판별했습니다. 하드웨어의 전원 차단과 펌웨어의 송신 조건을 함께 적용했습니다.

> **인증 실패 처리**
> HC-05가 꺼진 상태에서는 무선 송신을 하지 않고, 로컬 LED로 실패를 표시하며 잠금을 유지합니다.

구현 02 · 입력 처리와 통신

ADC Circular DMA

2축 조이스틱 값을 연속 수집하고 입력 처리와 분리했습니다.

EMA · 중심 보정 · Dead Zone · Clamp

노이즈와 중심 오차를 줄이고 중립 구간 및 출력 범위를 정했습니다.

GPIO EXTI · 디바운싱

인터럽트에서는 이벤트만 전달하고, 일반 실행 흐름에서 스위치 상태를 판정했습니다.

FreeRTOS · UART 패킷 송신

인증·입력·송신 역할을 분리하고 차량 담당자와 명령 형식을 맞췄습니다.

통합 결과 등록·미등록 카드, 중립·방향 입력, 스위치 반복 조작을 확인하고 Controller 명령이 실제 차량 동작으로 이어지는 것까지 검증했습니다.

PLC 자동 적재·출고 시스템

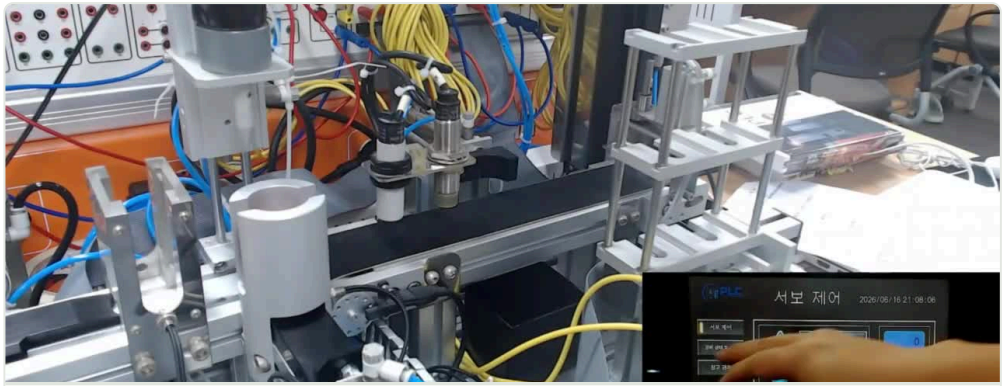
금속·비금속 소재를 분류해 보관하고, 조건에 맞춰 자동으로 배출하는 공정입니다.

소재를 공급·이송해 6개 위치에 적재하고, 소재 종류와 저장 위치를 기억합니다. 같은 소재가 3개 모이면 자동 출고하며, 작업자가 선택·전체 배출도 할 수 있습니다.

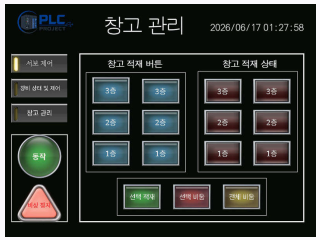
나의 역할

HMI 화면 구성 · 배출 시퀀스 구현

창고 상태를 확인하고 배출을 지시하는 HMI와, 해당 명령을 실제 설비 동작으로 연결하는 PLC 로직을 담당했습니다.



전체 자동화 설비



직접 구성한 HMI · 상태 표시와 배출 명령

직접 구현한 내용

- **HMI 화면 구성**
  적재 위치와 소재 종류를 표시하고 선택 비움·전체 비움 조작 화면을 구현했습니다.
- **기본 · 선택 · 전체 배출**
  기본 동작 완료 후 선택 배출을 수행하고, 전체 배출에서는 빈 위치를 건너뛰도록 구성했습니다.
- **배출 순서 · 인터록**
  서보 이동·흡착·배출 순서를 제어하고, 배출 중 다른 배출 명령이 끼어들지 않게 차단했습니다.

소재 공급 → 금속·비금속 분류 → 위치별 적재 → **배출 모드 선택** → **서보 이동 · 흡착 · 배출**

Mitsubishi PLC | GX Works2 | GT Designer3 | Ladder | Servo | Interlock